

GRAPHFLOW AI

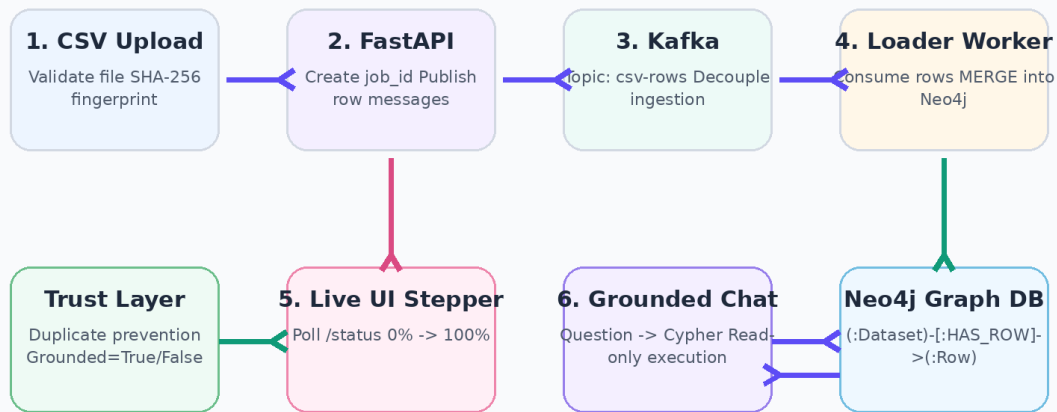
CSV -> Kafka -> Neo4j -> Grounded Chat

Technical Architecture & Demo Documentation

GraphFlow AI converts arbitrary CSV tabular data into a queryable Neo4j graph and answers natural-language questions using safe, dataset-grounded Cypher queries.

GraphFlow AI - Data-to-Graph Architecture

Asynchronous CSV ingestion + Neo4j graph construction + grounded natural-language querying



Architecture Overview

1. Executive Summary

GraphFlow AI is an end-to-end data ingestion and question-answering platform designed for hackathon-scale demonstrations and extensible production architecture. A user uploads a CSV file, FastAPI validates and fingerprints the dataset, Apache Kafka streams row-level messages asynchronously, a loader worker constructs a graph in Neo4j, and a grounded chatbot converts natural-language questions into safe, read-only Cypher queries.

Core value proposition: fast ingestion, duplicate-safe graph construction, live progress visibility, and anti-hallucination answers that stay inside the uploaded dataset.

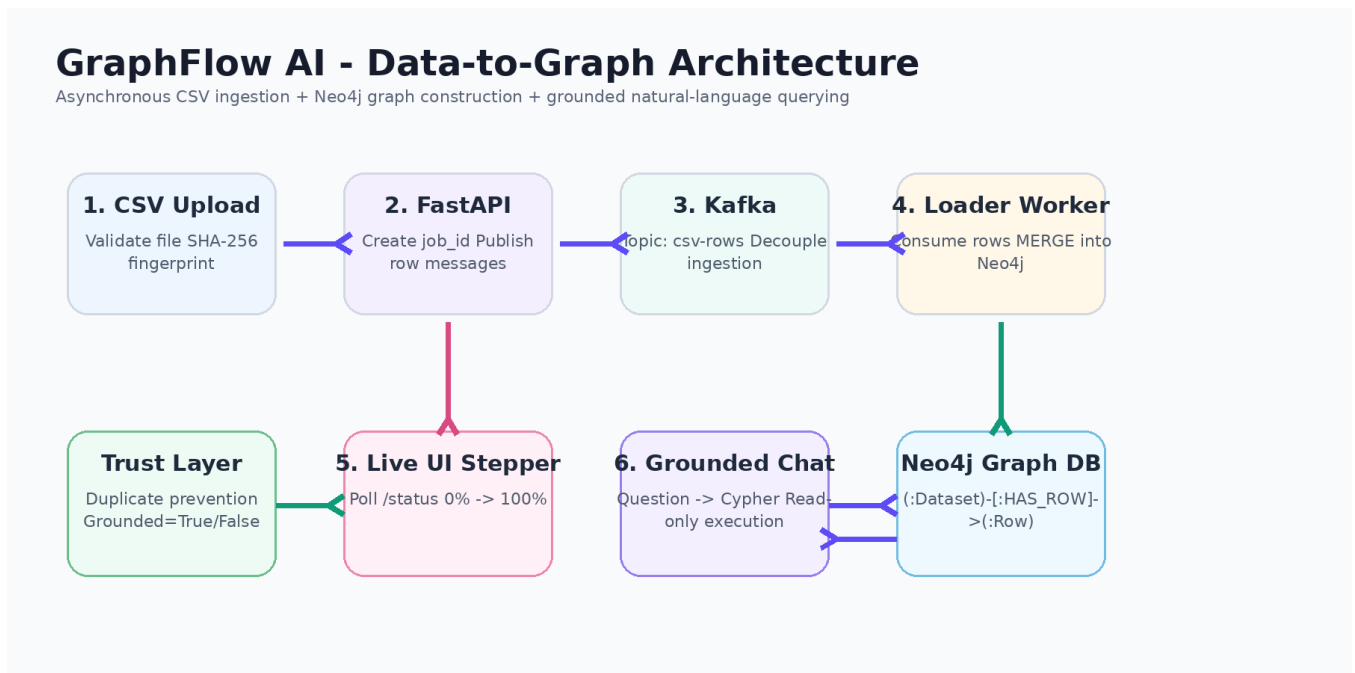
Problem It Solves

- Tabular CSV data is easy to store but difficult for non-technical users to explore using database query languages.
- Direct synchronous database imports can become slow or fragile as file size grows.
- Generic chatbots may invent answers when the requested field does not exist in the source data.
- Repeated uploads can create duplicate records unless the ingestion pipeline is idempotent.

Solution in One Line

Upload CSV -> Stream rows through Kafka -> Build Neo4j graph -> Ask questions in English -> Get only grounded answers.

2. End-to-End System Workflow



1. CSV File Drop - User drags and drops a CSV file into the web dashboard. The UI validates basic file type/size rules and sends the file to the FastAPI ingestion endpoint.

2. FastAPI Ingestion - The server parses the file, calculates a SHA-256 dataset fingerprint, creates a unique job_id, profiles the columns, and prepares row messages.

3. Kafka Streaming - Each row is published as a JSON message to the csv-rows topic. This separates request handling from database writes and improves resilience.

4. Loader Worker - An independent consumer reads Kafka messages and performs Neo4j MERGE operations using stable keys for the dataset and each row.

5. Live Progress - The frontend queries the job status endpoint and updates a visual stepper from upload through graph completion.

6. Grounded Chat - The chatbot inspects the active schema, generates read-only Cypher, executes it against Neo4j, verifies the result, and returns a natural-language answer.

3. Step 1 - CSV Upload & Validation (Data In)

1. User selects or drags a CSV file such as customers.csv or employees.csv into the dashboard.
2. Frontend submits the file to POST /ingest as multipart form data.
3. FastAPI parses headers and rows, checks for malformed CSV content, and builds a basic dataset profile.
4. The server computes a SHA-256 hash from the file bytes. This becomes the stable dataset_id.
5. A unique job_id is created for tracking progress independently from the dataset_id.

Recommended Metadata Stored for Each Job

Field	Purpose
job_id	Tracks the current ingestion run and UI progress.
dataset_id	SHA-256 fingerprint used to identify the CSV content.
filename	Original file name for display and audit.
total_rows	Used to calculate processing percentage.
processed_rows	Incremented by the loader worker.
status	queued / streaming / ingesting / completed / failed
created_at	Timestamp for traceability and sorting.

```
POST /ingest
Content-Type: multipart/form-data

Response:
{
  "job_id": "daae085d",
  "dataset_id": "<sha256>",
  "status": "queued"
}
```

4. Step 2 - Asynchronous Messaging with Apache Kafka

FastAPI converts each parsed CSV row into a compact JSON message and publishes it to a Kafka topic such as **csv-rows**. The API is no longer responsible for waiting until Neo4j finishes every write.

```
{
  "job_id": "daae085d",
  "dataset_id": "<sha256>",
  "row_index": 1,
  "data": {
    "Name": "Arun",
    "City": "Chennai",
    "Salary": 55000
  }
}
```

Why Kafka?

- Decouples file ingestion from database persistence.
- Provides buffering when database writes are slower than file parsing.
- Supports retry/replay patterns and future horizontal scaling with multiple consumers.
- Makes pipeline stages independently observable and replaceable.

Technical note: For a very small MVP, direct background processing can work. Kafka is included here because the project explicitly demonstrates event-driven, scalable ingestion architecture.

5. Step 3 - Graph Construction in Neo4j

A continuously running loader worker consumes messages from Kafka and writes them into Neo4j using the official Bolt driver. The graph model stays intentionally generic so it can accept arbitrary CSV schemas.

```
(:Dataset {id: dataset_id, filename: "customers.csv"})
|
+--[:HAS_ROW]--> (:Row {dataset_id: dataset_id, row_index: 1, Name: "Arun", City:
"Chennai", ...})
```

Example Cypher MERGE Pattern

```
MERGE (d:Dataset {id: $dataset_id})
SET d.filename = $filename

MERGE (r:Row {dataset_id: $dataset_id, row_index: $row_index})
SET r += $row_data

MERGE (d)-[:HAS_ROW]->(r)
```

Idempotency guarantee: duplicate prevention is achieved only when MERGE uses stable keys and the database has appropriate uniqueness constraints. Re-uploading identical content reuses the same dataset_id instead of creating a second logical dataset.

```
CREATE CONSTRAINT dataset_id_unique IF NOT EXISTS
FOR (d:Dataset) REQUIRE d.id IS UNIQUE;

CREATE CONSTRAINT row_identity_unique IF NOT EXISTS
FOR (r:Row) REQUIRE (r.dataset_id, r.row_index) IS UNIQUE;
```

6. Step 4 - Real-Time Status & UI Stepper

The frontend displays the pipeline as a live sequence of stages. In the MVP, the browser polls a status endpoint approximately every 500 ms to 1 second. A later version can replace polling with Server-Sent Events or WebSockets.

Stage	Example Status	UI Feedback
Upload	complete	Green check mark
Kafka	streamed	Green check mark
Neo4j Ingestion	processing	Animated progress bar
Completion	100%	Success state + dataset preview
Failure	failed	Readable error + retry action

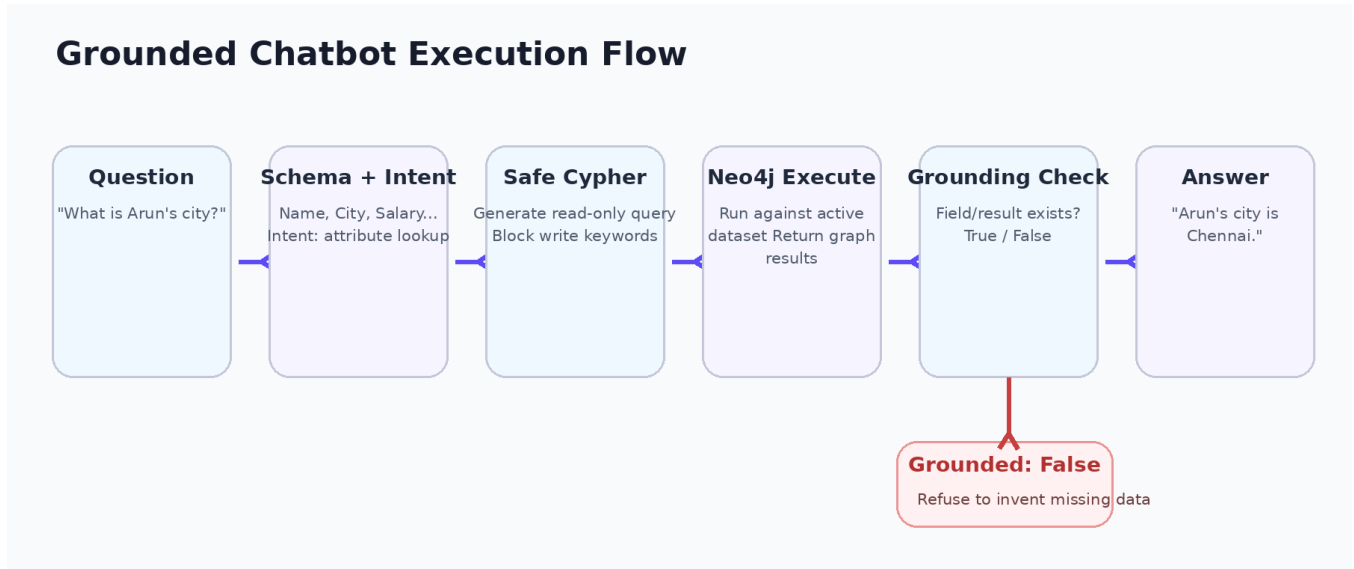
```
GET /status?job_id=daae085d
```

```
{
  "status": "ingesting",
  "total_rows": 10000,
  "processed_rows": 7425,
  "progress": 74.25
}
```

Dataset Profile & Preview Card

- Row count and column count
- Column names and inferred data types
- Missing-cell counts or percentages
- First five rows as a preview table
- Dataset hash / ID and ingestion status

7. Step 5 - Grounded Natural-Language Chatbot



When a user asks a question, the chatbot must answer from the active Neo4j dataset rather than from general model memory. The pipeline therefore separates **language understanding** from **database truth**.

1. **Question Input** - POST /chat receives the question and active dataset_id.
2. **Schema Awareness** - The engine reads available CSV-derived properties/columns and uses them as the allowed vocabulary.
3. **Intent Parsing** - The request is classified into operations such as row count, filtering, lookup, sum, average, min/max, or grouping.
4. **Cypher Generation** - A query is produced using only allowed labels, properties, parameters, and read-only clauses.
5. **Safety Validation** - Reject queries containing mutating clauses such as CREATE, DELETE, DETACH, SET, REMOVE, DROP, or procedure calls not on an allowlist.
6. **Database Execution** - Run the query against the active dataset in Neo4j.
7. **Grounding Verification** - If the requested field is absent or the query produces no supported fact, return Grounded: False.
8. **Natural Answer** - Convert database results into a short human-readable answer and optionally display the Cypher used.

```

MATCH (r:Row {dataset_id: $dataset_id})
WHERE toLower(toString(r.Name)) CONTAINS toLower($name)
RETURN r.City AS city
LIMIT 5
  
```

Grounded: True The answer is directly supported by a property/result returned from Neo4j.

Grounded: False The requested field or evidence is unavailable. Example: 'I don't have information about blood group in the uploaded data.'

8. Suggested API Contract

Method	Endpoint	Purpose	Key Output
POST	/ingest	Upload and validate CSV	job_id, dataset_id
GET	/status	Read ingestion progress	status, progress
GET	/dataset/{id}	Dataset profile and preview	schema, row count, sample rows
POST	/chat	Ask grounded question	answer, cypher, grounded
GET	/graph/{id}	Graph visualization payload	nodes, relationships

9. Key Features for the Hackathon Demo

Feature	Why It Matters
Drag-and-drop CSV upload	Simple user entry point with instant preview.
SHA-256 duplicate fingerprint	Detects exact file re-uploads.
Kafka event streaming	Shows modern asynchronous architecture.
Neo4j generic graph model	Works with different CSV schemas without redesigning tables.
Live pipeline stepper	Makes backend activity visible and demo-friendly.
Dataset profiling	Rows, columns, missing values, preview.
Natural-language data Q&A	Removes need to know Cypher.
Cypher transparency	Shows the generated query to build trust.
Grounded badge	Clearly indicates whether an answer is supported by data.
Duplicate prevention	MERGE + uniqueness constraints keep graph clean.
Error handling	Readable states for invalid CSV, Kafka failure, or DB error.
Suggested questions	Helps judges immediately test the chatbot.
Graph visualization	Shows Dataset -> Row relationships interactively.

10. Technology Stack

Layer	Technology	Role
Frontend	React / Vite	Upload UI, stepper, preview, chatbot, graph visualization
API	FastAPI (Python)	CSV parsing, hashing, job creation, REST endpoints
Streaming	Apache Kafka	Asynchronous row messaging and buffering
Worker	Python consumer	Kafka consumption and Neo4j loading
Database	Neo4j	Graph storage and Cypher query execution
LLM / NLP	LLM + deterministic intent rules	Natural language to safe Cypher /

		answer formatting
Hashing	SHA-256	Stable dataset fingerprint
Deployment	Docker Compose	Run services consistently during demo

11. Recommended 90-Second Demo Flow

1. Upload customers.csv and immediately show the file preview, row/column profile, and generated dataset fingerprint.
2. Point to the live stepper as rows move through Kafka and the Neo4j ingestion percentage increases.
3. Open the graph view and show the Dataset node connected to Row nodes.
4. Ask: "How many rows are there?" and show the generated Cypher plus Grounded: True.
5. Ask: "What is Arun's city?" and show the exact data-backed answer.
6. Ask for a nonexistent field such as blood group and demonstrate Grounded: False instead of hallucination.
7. Re-upload the exact CSV and show that duplicate nodes are not created.

12. Judge-Ready Explanation

"GraphFlow AI ingests arbitrary CSV tabular data, fingerprints the dataset with SHA-256, streams each row asynchronously through Apache Kafka into Neo4j, and converts natural-language questions into safe, read-only Cypher queries. Every answer is verified against the graph, so the system can explicitly say when information is missing instead of hallucinating."

Why This Architecture Stands Out

- Clear separation of concerns: API, stream, worker, database, and chatbot are independent components.
- Scalable design: Kafka and consumer workers allow ingestion throughput to grow without changing the UI.
- Trustworthy AI: answers are tied to query results and the active dataset schema.
- Demo visibility: progress, generated Cypher, graph structure, and grounding status make the system easy to judge.
- Data integrity: stable IDs, MERGE operations, and uniqueness constraints support idempotent ingestion.

13. Production-Readiness Notes

- Use parameterized Cypher instead of string-concatenating user inputs.

- Use an allowlist of supported read-only Cypher patterns rather than only a keyword blacklist.
- Scope every query by dataset_id so data from two uploads cannot mix.
- Store Kafka offsets / processing state carefully so retries do not corrupt progress counts.
- Limit CSV size, row length, and allowed property names; normalize unsafe headers before writing them to Neo4j.
- Add authentication and per-user dataset ownership before any multi-user deployment.
- For large datasets, batch Kafka messages or use Neo4j UNWIND transactions for better write throughput.

GRAPHFLOW AI

From CSV rows to a trustworthy graph conversation.